

Lab4_ML_KNN

September 16, 2026

0.1 K-Nearest Neighbour

K-Nearest Neighbors (KNN) is a supervised learning algorithm used for classification and regression tasks. It is a non-parametric, instance-based learning method that classifies new data points based on their similarity to the nearest neighbors.

Advantages

Simple and easy to implement. No training phase (lazy learning). Works well for lower-dimensional data. Can handle multi-class classification.

Disadvantages

Computationally expensive for large datasets (since it searches all points). Memory-intensive (stores all training data).

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

```
[20]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
[22]: df = pd.read_csv('Plant_Tree_data.csv')
df.shape
```

```
[22]: (17898, 9)
```

```
[23]: df.columns
```

```
[23]: Index(['LeafLength', 'LeafBreadth', 'ErrorLeafLength', 'ErrorLeafBreadth',
        'LeafCurveAngle', 'EdgeCurvatureAngle', 'ErrorLeafCurvature',
        'TreeLength', 'TreeType'],
        dtype='object')
```

```
[24]: df.head()
```

```
[24]:   LeafLength  LeafBreadth  ErrorLeafLength  ErrorLeafBreadth  LeafCurveAngle \
0   140.562500    55.683782         -0.234571         -0.699648         3.199833
1   102.507812    58.882430          0.465318         -0.515088         1.677258
2   103.015625    39.341649          0.323328          1.051164         3.121237
3   136.750000    57.178449         -0.068415         -0.636238         3.642977
```

4	88.726562	40.672225	0.600866	1.123492	1.178930
	EdgeCurvatureAngle	ErrorLeafCurvature	TreeLength	TreeType	
0	19.110426	7.975532	74.242225	0	
1	14.860146	10.576487	127.393580	0	
2	21.744669	7.735822	63.171909	0	
3	20.959280	6.896499	53.593661	0	
4	11.468720	14.269573	252.567306	0	

```
[26]: df.TreeType.unique()
```

```
[26]: array([0, 1])
```

```
[45]: # from sklearn.preprocessing import LabelEncoder

# lable_encode = LabelEncoder()
# df['species'] = lable_encode.fit_transform(df['species'])
# df.head()
```

```
[27]: # Separate independent and dependent variables
X = df.iloc[:, :-1].values
y = df.iloc[:, 8].values
```

```
[28]: print(type(X))
print(X.shape)
print(y.shape)
```

```
<class 'numpy.ndarray'>
(17898, 8)
(17898,)
```

```
[31]: #Split data into training and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20,
↳random_state=10)
X_train.shape
```

```
[31]: (14318, 8)
```

```
[32]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

```
[33]: from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors = 3,p=2, metric='minkowski') #↳
↳default n_neighbors are 5
```

```
classifier.fit(X_train, y_train)
```

```
[33]: KNeighborsClassifier(n_neighbors=3)
```

```
[34]: y_pred = classifier.predict(X_test)
```

```
[35]: y_pred
```

```
[35]: array([0, 0, 0, ..., 0, 0, 0], shape=(3580,))
```

0.2 Confusion matrix

A Confusion matrix is an $N \times N$ matrix used for evaluating the performance of a classification model, where N is the total number of target classes. The matrix compares the actual target values with those predicted by the machine learning model.

```
[36]: from sklearn.metrics import classification_report, confusion_matrix, \
      ↪ accuracy_score

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
cr = classification_report(y_test, y_pred)
print("\nClassification Report:",)
print(cr)
acc = accuracy_score(y_test, y_pred)
print("\nAccuracy:", acc)
```

Confusion Matrix:

```
[[3242  30]
 [ 67 241]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.99	0.99	3272
1	0.89	0.78	0.83	308
accuracy			0.97	3580
macro avg	0.93	0.89	0.91	3580
weighted avg	0.97	0.97	0.97	3580

Accuracy: 0.9729050279329609

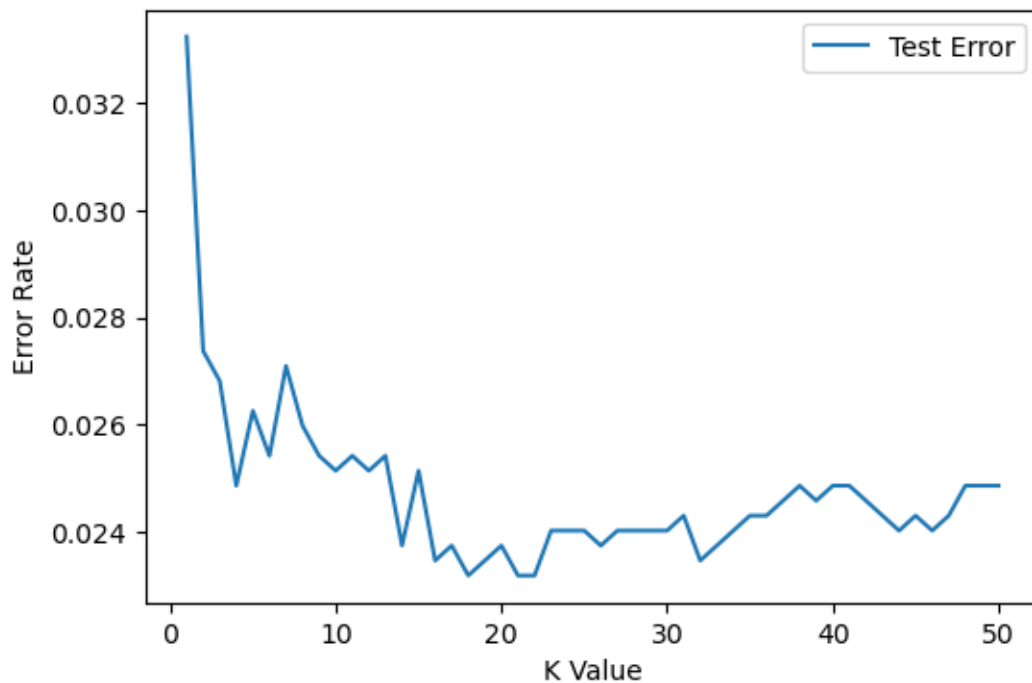
```
[39]: test_errors = []

      for i in range(1, 51):
```

```
knn = KNeighborsClassifier(n_neighbors=i,p=3, metric='minkowski')
knn.fit(X_train, y_train)
pred_i = knn.predict(X_test)
test_errors.append(np.mean(pred_i != y_test))
```

```
[40]: plt.figure(figsize=(6,4),dpi=100)
plt.plot(range(1, 51),test_errors,label='Test Error')
plt.legend()
plt.ylabel('Error Rate')
plt.xlabel("K Value")
```

```
[40]: Text(0.5, 0, 'K Value')
```



0.3 Task 1

You are required to fit a regressor on the Food_delivery_times dataset Dataset description: The dataset includes several independent variables and a dependent variable.

Now, fit a KNN regression model and find the optimal k value using the elbow approach.

0.4 Task 2

dataset: Food_delivery_times dataset

Perform k-fold cross-validation to find the optimal k and p values

$k \in \{1, 3, 5, 7, 9, 11, 13, 15\}$ and $p \in \{1, 2, 3\}$

Consider k values in k -fold cross-validation as 5.

Documentation: https://scikit-learn.org/stable/modules/cross_validation.html

[]: